Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Estructura de Datos

Sección: A

Docente: Mario Moisés Ramírez Tobar

“TAD’s de Estructuras Proyecto 1 Fase 1”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 03 de abril de

2026

TAD's Proyecto 1 Fase 1

1. TAD Producto (Struct)

Estructura que representa un producto del supermercado.

Dominio:

⟨nombre, codigoBarras, categoria, fechaVencimiento, marca, precio, stock⟩

Operaciones:

Constructor Producto()

Prototipo: Producto()

Complejidad: O(1)

Constructor parametrizado

Prototipo: Producto(string n, string cb, string cat, string fv, string m, double p, int s)

Complejidad: O(1)

2. TAD NodoListaEnlazada

Elemento para la construcción de listas dinámicas.

Dominio:

⟨valor, siguiente⟩ donde:

valor: Elemento de tipo genérico T

siguiente: Puntero a otro NodoListaEnlazada<T> o nullptr

Operaciones:

Constructor NodoListaEnlazada()

Prototipo: NodoListaEnlazada(const T& valor)

Complejidad: O(1)

getValor(), setValor()

Complejidad: O(1)

getSiguiente(), setSiguiente()

Complejidad: O(1)

3. TAD ListaEnlazada

Estructura lineal dinámica donde cada elemento contiene un puntero al siguiente. Se utiliza para búsqueda secuencial (comparativa).

Dominio:

Secuencia de elementos de tipo T, con head y size

Operaciones:

Constructor ListaEnlazada()

Complejidad: O(1)

insertarAlFinal()

Prototipo: void insertarAlFinal(const T& valor)

Complejidad: O(n) - Recorre hasta el final

insertarOrdenadoPorNombre()

Prototipo: void insertarOrdenadoPorNombre(const T& valor)

Complejidad: O(n) - Busca posición de inserción

buscarPorCodigo()

Prototipo: T* buscarPorCodigo(std::string codigo)

Complejidad: O(n) - Recorrido secuencial

eliminarPorCodigo()

Prototipo: bool eliminarPorCodigo(std::string codigo)

Complejidad: O(n)

getSize(), isEmpty()

Complejidad: O(1)

4. TAD NodoArbolAVL

Elemento para la construcción del árbol AVL.

Dominio:

⟨valor, izq, der, fe⟩ donde:

valor: Producto

izq: NodoArbolAVL* - Hijo izquierdo

der: NodoArbolAVL* - Hijo derecho

fe: int - Factor de equilibrio

Constructor NodoArbolAVL()

Prototipo: NodoArbolAVL(const Producto& p)

Complejidad: O(1)

getValor(), getIzq(), getDer(), getFe()

Complejidad: O(1)

setValor(), setIzq(), setDer(), setFe()

Complejidad: O(1)

5. TAD ArbolAVL

Árbol binario de busqueda auto-balanceado. Ordenado por nombre del producto. La diferencia de alturas entre subarboles no puede ser mayor a 1.

Dominio:

⟨raiz⟩

Operaciones:

insertar()

Prototipo: bool insertar(const Producto& p)

Complejidad: O(log n)

buscar()

Prototipo: Producto* buscar(const std::string& nombre)

Complejidad: O(log n)

eliminar()

Prototipo: bool eliminar(const std::string& nombre)

Complejidad: O(log n)

rotaciones (II, DD, ID, DI)

Complejidad: O(1) cada una

6. TAD NodoArbolB

Pagina del arbol B. Puede contener múltiples llaves y punteros a hijos.

Dominio:

⟨llaves[MAX_LLAVES], hijos[MAX_HIJOS], numLlaves, esHoja⟩

Constantes:

ORDEN_B = 2

MAX_LLAVES = 4

MAX_HIJOS = 5

Operaciones:

Constructor NodoArbolB()

Prototipo: NodoArbolB(bool hoja = true)

Complejidad: O(1)

getLlave(), getHijo(), getNumLlaves(), getEsHoja()

Complejidad: O(1)

estaLleno()

Complejidad: O(1)

7. TAD ArbolB

Arbol B de orden 2. Ordenado por fecha de vencimiento. Ideal para búsquedas por rango.

Dominio:

⟨raiz⟩

Operaciones:

insertar()

Prototipo: bool insertar(const Producto& p)

Complejidad: O(log n)

buscar()

Prototipo: Producto* buscar(const std::string& fecha)

Complejidad: O(log n)

buscarPorRango()

Prototipo: void buscarPorRango(const std::string& desde, const std::string& hasta)

Complejidad: O(log n + k) donde k = resultados

eliminar()

Prototipo: bool eliminar(const std::string& fecha)

Complejidad: O(log n)

split() (privado)

Complejidad: O(ORDEN_B) = O(1)

8. TAD NodoArbolBPlus

Nodo del arbol B+. Similar al nodo B, pero con puntero adicional para enlazar hojas.

Dominio:

⟨llaves[MAX_LLAVES_BP], hijos[MAX_HIJOS_BP], numLlaves, esHoja, siguiente⟩

Operaciones adicionales:

getSiguiente(), setSiguiente()

Complejidad: O(1)

9. TAD ArbolBPlus

Arbol B+ de orden 2. Todas las llaves estan en las hojas, y las hojas están enlazadas. Ordenado por categoria.

Dominio:

⟨raiz⟩

Operaciones:

insertar()

Prototipo: void insertar(const Producto& p)

Complejidad: O(log n)

buscarPorCategoria()

Prototipo: void buscarPorCategoria(const std::string& categoria)

Complejidad: O(log n + k)

Ventaja: Hojas enlazadas → recorrido eficiente

eliminar()

Prototipo: bool eliminar(const std::string& categoria, const std::string& codigo)

Complejidad: O(log n)

10. TAD TablaHash

Estructura que permite busquedas por clave única con tiempo promedio O(1). Implementa encadenamiento con ListaEnlazada.

Dominio:

⟨tabla[TAMANIO_INICIAL]⟩ donde cada posición es una ListaEnlazada<Producto>

Constantes:

TAMANIO_INICIAL = 101 (número primo)

Operaciones:

funcionHash() (privada)

Algoritmo: djb2

Complejidad: O(k) donde k = longitud del código

insertar()

Prototipo: void insertar(const Producto& p)

Complejidad: O(1) promedio

buscar()

Prototipo: Producto* buscar(std::string codigo)

Complejidad: O(1) promedio

eliminar()

Prototipo: void eliminar(std::string codigo)

Complejidad: O(1) promedio

11. TAD Pila

Estructura LIFO utilizada para el rollback (deshacer eliminaciones).

Dominio:

⟨tope, tamanio⟩

Operaciones:

push()

Prototipo: void push(const T& valor)

Complejidad: O(1)

pop()

Prototipo: T pop()

Complejidad: O(1)

peek()

Complejidad: O(1)

estaVacia()

Complejidad: O(1)

Tabla Resumen de Complejidades

TAD	Operación	Complejidad
Producto	Constructor	O(1)
NodoListaEnlazada	Constructor / Getters / Setters	O(1)
ListaEnlazada	insertarAlFinal / insertarOrdenado	O(n)
ListaEnlazada	buscarPorCodigo / eliminarPorCodigo	O(n)
NodoArbolAVL	Constructor / Getters / Setters	O(1)
ArbolAVL	insertar / buscar / eliminar	O(log n)
NodoArbolB	Constructor / Getters / estaLleno	O(1)
ArbolB	insertar / buscar / eliminar	O(log n)
ArbolB	buscarPorRango	O(log n + k)
NodoArbolBPlus	Constructor / Getters / Setters	O(1)
ArbolBPlus	insertar / buscar / eliminar	O(log n)
ArbolBPlus	buscarPorCategoria	O(log n + k)
TablaHash	insertar / buscar / eliminar	O(1) promedio
Pila	push / pop / peek	O(1)